

Universidad de Costa Rica

I Semestre - 2026

Escuela de Ciencias de la Computación e Informática

CI-0123 P.I. de Redes y Sistemas Operativos

Docentes:

Francisco Arroyo

Maeva Murcia

Migración de protocolos

Juan José González Márquez - C4F588

Juan Loaiza - B74200

Ariana Ardilla Gutierrez - C4C646

Jeferson Marín - C24549

Migración a Uppercase v3.0

Durante el desarrollo del proyecto fue necesario migrar del protocolo original utilizado por el sistema a un nuevo protocolo llamado Uppercase v3.0. El objetivo principal de esta migración es simplificar la comunicación entre cliente, intermediario y servidor, utilizando mensajes de texto fáciles de procesar y mantener.

El protocolo anterior trabajaba con estructuras más complejas, utilizando tipos numéricos, paquetes binarios y varios campos específicos. En cambio, el nuevo protocolo utiliza únicamente mensajes de texto con el siguiente formato:

P/[COMANDO]/[MENSAJE]

Por esta razón es necesario realizar los siguientes cambios en el programa:

1. Cambiar la estructura de los mensajes

El cambio más importante es que ya no se utilizan paquetes binarios ni estructuras complejas.

Antes los mensajes tenían varios campos como:

- tipo
- segmento
- ip source
- content_length
- piezas

Ahora todo se envía como un string simple. Por lo tanto, se debe eliminar toda la lógica relacionada con la serialización binaria.

2. Implementar un parser de strings

Ahora el programa debe interpretar mensajes separados por (/).

3. Reemplazar los tipos numéricos por comandos

El protocolo viejo utilizaba enums y números para identificar mensajes.

Ahora se utilizan letras:

Comando	Función
C	Connect
R	Request
G	Get
D	Data
Q	Quit

F

Fork

K

Kill

4. Eliminar la lógica de segmentos

El protocolo anterior trabajaba con mitades de figuras:

- mitad 1
- mitad 2
- figura completa

El nuevo protocolo ya no utiliza segmentos.

Entonces se debe eliminar:

- variables `segment`
- `half`
- validaciones `INVALID_HALF`
- lógica de división de figuras

5. Cambiar la forma de enviar datos

Antes se enviaban estructuras completas con `send( )`, ahora simplemente se envía un string.

6. Actualizar los comandos del cliente

El cliente debe cambiar los mensajes que envía:

> Solicitar directorio

Antes:

LIST_FIGURES

Ahora:

P/R/dir

> Solicitar una figura

Antes:

GET_FIGURE

Ahora:

P/G/nombreFigura

>Cerrar conexión

Ahora se utiliza:

P/Q/

7. Cambiar las respuestas del servidor

El servidor ahora responde utilizando el comando D.

8. Cambiar el descubrimiento de servidores

El protocolo viejo utilizaba JOIN y HANDSHAKE. Ahora el fork hace broadcast UDP utilizando: P/C/ip:puerto

Los servidores escuchan el broadcast y luego establecen conexión TCP con el fork.

9. Modificar las conexiones entre sockets

Antes las conexiones eran persistentes, ahora la conexión funciona así:

1. conectar
2. enviar mensaje
3. recibir respuesta
4. enviar QUIT
5. cerrar socket

Esto significa que ya no es necesario mantener tantos sockets abiertos permanentemente.

10. Actualizar la comunicación entre forks

Antes se utilizaban paquetes especializados entre intermediarios, ahora se utiliza: P/F/fig1,fig2,fig3 en broadcast UDP.

Esto obliga a cambiar la lógica de sincronización de figuras entre forks.

11. Cambiar las validaciones

Antes se validaban:

- tamaños
- tipos numéricos
- content_length
- segmentos

Ahora únicamente debe validarse:

- que el mensaje empiece con P/
- que tenga 3 partes
- que el comando exista